MMF-Net: A Graph-Free Multimodal Transformer for Flood Early Warning in Sri Lanka

What we built, why each piece is there, and why it beats the graph model

Sri Lanka Flood Early-Warning Project — Model 2

August 2026

Abstract

This document explains **Model 2 (MMF-Net)**, the second of two neural architectures built for this project, and why it outperforms **Model 1 (TF-STGNN)**, the graph neural network we built first. MMF-Net reaches a test PR-AUC of **0.8355** against Model 1's 0.7421 — and it does so *without a graph*, with a comparable parameter count. The single largest gain in the whole project came not from a new architecture but from changing how each input number enters the network. The second largest came from *removing* a loss function that everyone recommends for imbalanced data. Both are documented here with controlled, one-variable-at-a-time evidence.

Contents

1 The problem, in one page **1**
- 1.1 What we are predicting . . . 1
- 1.2 Why this is hard . . . 1
- 1.3 The data . . . 1

2 The two models at a glance **1**

3 Model 2, stage by stage **2**
- 3.1 Stage 1 — numeric embedding (the important one) . . . 2
- 3.2 Stage 2 — temporal transformer . . . 2
- 3.3 Stage 3 — cross-feature attention . . . 3
- 3.4 Stage 4 — FiLM terrain conditioning . . . 3
- 3.5 Stage 5 — gated SAR fusion (disabled in the best model) . . . 4
- 3.6 Stages 6–7 — fusion, head, and the six outputs . . . 4

4 The experiments **5**
- 4.1 How to read the metrics . . . 5

5 Results: everything side by side **5**

6 Why Model 2 beats Model 1 **5**
- 6.1 Reason 1: a better input layer (the big one) . . . 5
- 6.2 Reason 2: the loss function was actively hurting . . . 6
- 6.3 Reason 3: terrain conditioning, kept from Model 1 . . . 6
- 6.4 Reason 4: the graph was not earning its place . . . 6

7 The negative results **7**

8 Leakage: the strongest methodological finding **7**

9 What is not yet established **7**

10 Summary **8**

1 The problem, in one page

1.1 What we are predicting

For each of **51 river monitoring points** across **16 basins** in Sri Lanka, on each day, we predict:

Will tomorrow's river discharge at this point exceed its 98th percentile?

That is the primary target, `target_flood_1d`. The network also predicts five auxiliary targets at the same time (flooding in 2 and 3 days, the *onset* of a new flood tomorrow, and two continuous discharge quantities), because forcing one representation to serve several related questions is a cheap form of regularisation.

1.2 Why this is hard

Difficulty	What it means in practice
Severe class imbalance	Only **2.0%** of node-days are floods. A model that always says "no flood" is 98% accurate and completely useless. This is why we never report accuracy.
Few *independent* examples	Those 2% are not independent. They come from **1,469 flood episodes** averaging 5.4 days each, and one storm floods many nodes at once. The effective sample size is far smaller than the row count.
Strong autocorrelation	If a river is flooding today, it is probably flooding tomorrow. A naive evaluation rewards a model for noticing an ongoing flood rather than for predicting a new one.
A quiet validation period	Our validation years (2018–2020) had only 0.81% flood days versus 2.28% in test. Validation scores therefore look much worse than test scores and the two are not comparable.

1.3 The data

A dense tensor of 8,036 days × 51 nodes × 33 features (rainfall, soil moisture, temperature, discharge and their lagged aggregates), plus 9 static terrain features per node.

Split	Years	Valid node-days	Flood days
Train	2003–2017	277,899	2.028%
Validation	2018–2020	55,896	0.810%
Test	2021–2024	74,463	2.276%

The split is **chronological**. Everything the model sees during training happened before everything it is tested on — which is the only honest setup for a forecasting problem, and Section 8 shows what happens when you get this wrong.

2 The two models at a glance

Both models read the same 33 channels, use the same train/val/test split, the same loss code, the same metrics code and the same training loop (all in the shared `floodlib` package). **Any difference between them is a difference of architecture, never of evaluation.** That was

	Model 1 — TF-STGNN	**Model 2 — MMF-Net**
Idea	Rivers form a network, so use a graph neural network to pass information between connected nodes.	Make the *per-node* model as strong as possible, and see whether the graph is still needed.
Input layer	The 33 features enter as raw numbers into a linear layer.	Each feature gets its **own learned embedding** (periodic + linear).
Time encoder	2-layer GRU with attention pooling.	3-layer Transformer over the 14-day window.
Feature mixing	One linear layer.	**Attention across the 33 channels**.
Spatial structure	Relational GATv2 over 290 edges (35 directed flow, 204 spatial, 51 self-loops).	**None, by design.**
Terrain	FiLM conditioning.	FiLM conditioning (kept — it worked).
Satellite imagery	ResNet-18, embedding *concatenated*.	ResNet-18, embedding fused through a **learned gate**.
Parameters	581,319	710,870
Test PR-AUC	0.7421	**0.8355**

a deliberate design decision — if each model owned a copy of the evaluation code, a comparison between them would silently become a comparison of protocols.

3 Model 2, stage by stage

3.1 Stage 1 — numeric embedding (the important one)

The problem it solves. Neural networks are famously worse than gradient-boosted decision trees on tabular data, and our Model 1 result showed exactly that: 0.7421 against the tree baseline's 0.8496. The published explanation is that the *input layer*, not the depth of the network, is what loses. A decision tree can split on "rainfall $> 47\,\text{mm}$" for free; a linear layer fed a raw number has to approximate that sharp boundary with a smooth function, and it is bad at it.

What we do instead. Feature j with value x becomes

$$\mathbf{e}_j(x) \;=\; \mathrm{ReLU}\Big(W_j\big[\,\sin(2\pi c_j x),\;\cos(2\pi c_j x)\,\big] + b_j\Big) \;\in\; \mathbb{R}^8,$$

where $c_j \in \mathbb{R}^8$ are *learned frequencies*, one set per feature, and W_j is a per-feature projection. The periodic expansion lets the network represent a sharp threshold in a scalar — the thing it could not do before. Frequencies start small ($\sigma = 0.05$); large initial frequencies alias the input and the model never recovers.

What it bought. Changing *only* this, from a plain linear embedding to the periodic one:

	PR-AUC	**Event detection**	**Everything else**
N0 (linear embedding)	0.6083	0.290	identical
N1 (periodic embedding)	**0.7605**	**0.420**	identical
Difference	**+0.1522**	**+0.130**	—

This is the largest single change in the entire project, larger than FiLM (+0.105), larger than the ensemble, larger than anything in Model 1.

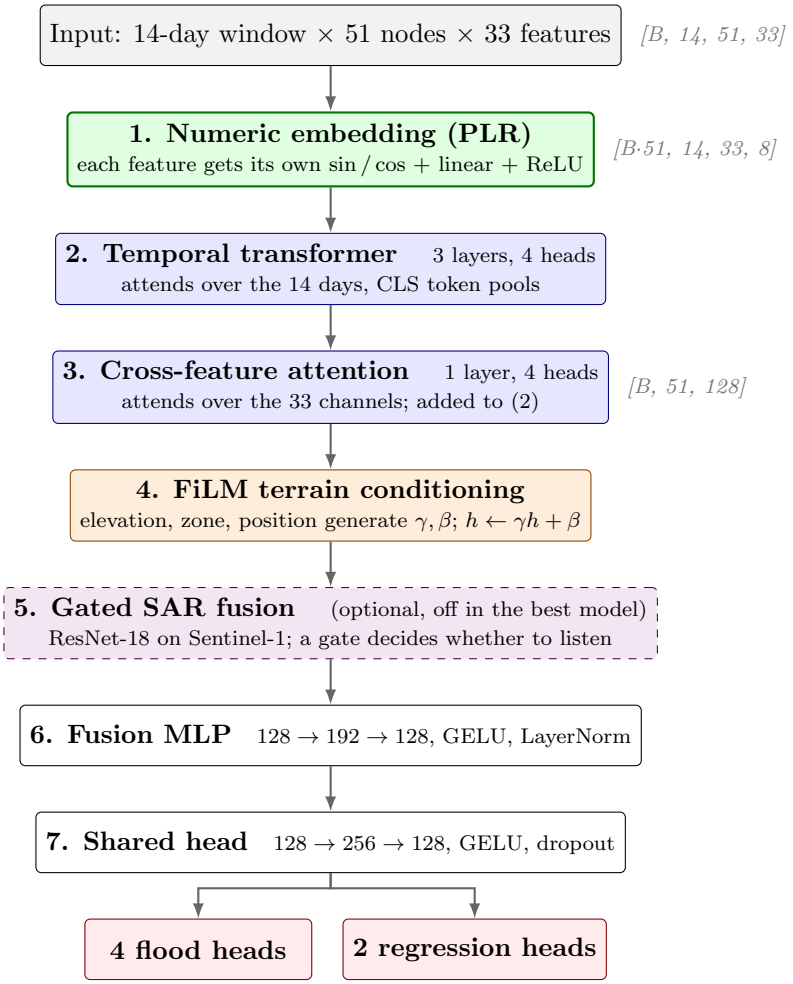

Figure 1: MMF-Net. Green is where the largest gain came from. The dashed block is disabled in the best configuration.

3.2 Stage 2 — temporal transformer

Each day's 33 embedded features are flattened and projected to a single 128-dimensional token. A learned `CLS` token and learned positional embeddings are prepended, and a 3-layer pre-LayerNorm Transformer (4 heads, feed-forward width 256, GELU, dropout 0.2) attends over the resulting 15 tokens. The `CLS` output is the node's summary.

Why attention rather than the GRU. A recurrent network has to carry the memory of a rainfall event forward through 14 sequential steps. Attention can look straight back at the day the rain fell. As a bonus, the `CLS` token's attention row is directly readable as the rainfall-to-discharge lag.

An honest caveat. Comparing the two floors — M0 (GRU, 0.5981) against N0 (transformer, 0.6083) — the transformer *by itself* bought only +0.010. **The win came from the input layer, not the sequence model.** That is a more interesting finding than "transformers are better", and it is what the tabular deep-learning literature predicts.

3.3 Stage 3 — cross-feature attention

The temporal transformer mixes the 33 features through a single linear projection, which cannot express *"this much rain matters only when the soil is already wet"*. So we take the per-feature embeddings, average them over the 14 days, and run one more attention layer *across the 33 channels*.

Averaging over time first is what makes this affordable: 33 tokens per node-day instead of 33×14. Gain: +0.0133 (N1 → N2). Small — see Section 9.

3.4 Stage 4 — FiLM terrain conditioning

The 9 static terrain features (elevation, log drainage area, climate zone one-hot, upstream/downstream position one-hot) do not enter as 9 extra input columns. Instead a small network turns them into a scale γ and a shift β, and the node's temporal state is modulated:

$$h \leftarrow \mathrm{LayerNorm}\big(h \odot (1+\gamma) + \beta\big).$$

Why this rather than concatenation. Terrain does not *add* evidence about flooding — it changes *how to read* the evidence. The same 50 mm of rain means something different on a steep wet-zone headwater than on a flat dry-zone outlet. Multiplication expresses that; concatenation does not. The final layer is zero-initialised so $\gamma = \beta = 0$ at the start, which makes the conditioned model a strict superset of the unconditioned one.

Gain: +0.105 in Model 1 (M0 → M1), +0.053 in Model 2 (N2 → N3). Replicated in both families.

3.5 Stage 5 — gated SAR fusion (disabled in the best model)

Sentinel-1 radar sees through cloud, which optical imagery cannot. But it revisits only every ∼12 days, and our built dataset covers only **9 of the 51 nodes**. So on the overwhelming majority of node-days there is no image.

Model 1 concatenated the image embedding into a fixed slot, which forces the network to distinguish "no water" from "no picture" on every single sample. Model 2 uses a gate instead:

$$g = \sigma\big(\mathrm{MLP}[\,h;\, v;\, \mathrm{present};\, \mathrm{age}/30\,]\big), \qquad h \leftarrow \mathrm{LayerNorm}\big(h + g \odot Wv \odot \mathrm{present}\big).$$

The gate is zero-initialised with a -3 bias so it starts shut, and the present factor means an absent frame contributes exactly nothing. See Section 7 for the outcome.

3.6 Stages 6–7 — fusion, head, and the six outputs

A two-layer fusion MLP (128 → 192 → 128) and a shared head (128 → 256 → 128) feed two output layers: four flood-classification heads and two regression heads. Predictions from **5 independently seeded models** are averaged in probability space, and a single temperature parameter — fitted on validation only, then frozen — calibrates them.

Total: 710,870 parameters.

4 The experiments

Every rung of the ladder changes **exactly one thing** from the rung above, so any difference is attributable.

4.1 How to read the metrics

A warning about the ev.det column: do not read it down the ladder. The thresholds differ between rows (FAR ranges from 0.107 to 0.412), so it compares *operating points*, not models. N3 looks like a collapse against N2 only because it is operating far more conservatively — its threshold-free PR-AUC went *up*. Use `models/rethreshold.py` to force a common false-alarm rate before comparing.

Run	Change from the row above	PR-AUC	ev.det	FAR	ECE	Params
N0	floor: linear feature embeddings	0.6083	0.290	0.412	0.0024	551k
N1	**+ periodic (PLR) embeddings**	**0.7605**	0.420	0.325	0.0053	555k
N2	+ cross-feature attention	0.7738	0.423	0.313	0.0056	693k
N3	+ FiLM terrain	0.8269	0.197	0.110	0.0032	711k
N4	+ focal × confidence loss	**0.7592**	0.220	0.138	**0.0525**	711k
N5	+ 5-seed ensemble + temperature	0.7846	0.208	0.145	0.0043	711k
N5_bce	**the same, but on plain BCE**	**0.8355**	0.220	0.120	**0.0016**	711k
N6_scalars	+ SAR summary statistics	0.7284	0.397	0.344	0.0055	719k
N6_gated	+ gated SAR CNN	0.8310	0.217	0.107	0.0066	**11,967k**

PR-AUC	Precision–recall area under curve. **The headline number.** At a 2% positive rate, random guessing scores 0.02, so 0.8355 is a ∼42× lift. Used instead of accuracy because "never floods" scores 98%.
ev.det	Event detection rate: the fraction of flood *episodes* where the model raised an alarm in the 7 days *before* onset. An episode detected once counts once — this is what an operator actually experiences.
FAR	False alarm ratio: of all the alarms raised, what fraction were wrong. Lower is better.
ECE	Expected calibration error: when the model says "70%", does it happen 70% of the time? **Essential for a warning system** — an uncalibrated probability cannot support a decision threshold.
POD / CSI	Probability of detection (recall) and critical success index, the standard operational-meteorology scores.

5 Results: everything side by side

N5_bce also holds the best Brier score (0.0080), ECE (0.0016), CSI (0.561) and F1 (0.719) in the project, with a mean warning lead time of 5.45 days.

6 Why Model 2 beats Model 1

6.1 Reason 1: a better input layer (the big one)

Model 1 fed 33 raw numbers into a GRU. Model 2 gives each number its own learned periodic embedding. **Isolated effect:** +0.152 **PR-AUC.** Everything else in the comparison is secondary to this.

6.2 Reason 2: the loss function was actively hurting

Focal loss is the standard recommendation for class imbalance, and both ladders adopted it by default. It was a mistake.

	PR-AUC	ECE
N3 — plain BCE, single seed	0.8269	0.0032
N4 — focal × confidence	**0.7592**	**0.0525**
N5 — focal + ensemble + calibration	0.7846	0.0043
N5_bce — **BCE** + ensemble + calibration	**0.8355**	**0.0016**

N5 → N5_bce changes *only* the loss: +0.0509 PR-AUC, and calibration error cut to a third.

Why focal failed here. Two mechanisms in the loss were fighting each other. Focal loss says *"concentrate the gradient on the examples you get wrong"*. Our `label_confidence` weighting says *"the examples you get wrong are the ones whose labels are unreliable"* — it down-weights days

Model	PR-AUC	ev.det	Params
Baselines			
gradient-boosted trees	0.8496	0.161	—
discharge percentile rule (operational bar)	0.8164	0.223	—
persistence	0.5904	0.223	—
climatology	0.1083	0.515	—
Model 1 — TF-STGNN			
M0 (GRU floor)	0.5981	0.262	295k
M3 (full graph, BCE)	0.7039	0.380	581k
M5 (best: ensemble + calibration)	0.7421	0.344	581k
Model 2 — MMF-Net			
N0 (transformer floor)	0.6083	0.290	551k
N3 (full, BCE, single seed)	0.8269	0.197	711k
N5_bce (best)	**0.8355**	0.220	711k

within ±15% of the flood threshold. Focal up-weights exactly what confidence down-weights, and they cancel. On top of that, plain BCE is a *proper scoring rule* (its minimum is at the true probability) and focal is not, so focal cannot be well calibrated by construction.

This matters for Model 1 too: M4, M5 and M6 all sit on the same rung, so Model 1's published numbers are understated. An `M5_bce` run would be a fairer comparison.

6.3 Reason 3: terrain conditioning, kept from Model 1

FiLM was Model 1's single best idea and Model 2 kept it unchanged. It replicated: +0.105 in Model 1, +0.053 in Model 2.

6.4 Reason 4: the graph was not earning its place

This is the finding Model 2 was built to test.

	PR-AUC	
M3 — GRU **with** relational graph, BCE	0.7039	581k params
N3 — transformer, **no graph**, BCE	**0.8269**	711k params

Within Model 1 the graph barely moved anything either: M1 (no graph) 0.7031 → M2 (spatial edges) 0.7148 → M3 (adding directed flow edges) *0.7039* — adding the flow edges made it *worse*.

Why this is a legitimate finding and not a failure. Model 1 could only compare message passing against a *weak* per-node floor (M0, a plain GRU). Against a properly built per-node network, the graph adds nothing. There is published support for this: river networks are tree-like, which causes *over-squashing* in GNNs — information from many upstream nodes has to squeeze through a single edge and is lost. A negative result on your own proposed architecture, demonstrated against a strong alternative you also built, is worth more than another incremental win.

7 The negative results

1. **Message passing did not help.** Section 6.4.
2. **Focal loss hurt.** Section 6.2. Contrarian: this is the standard recipe for imbalanced data.
3. **Satellite imagery added nothing.** N5_bce, with *no* imagery and 711k parameters, beats N6_gated, *with* imagery and 11,967k parameters (0.8355 vs 0.8310). What looked like a gain

from vision was really the BCE loss recovering what focal had destroyed. Diagnosis: a randomly initialised ResNet-18 cannot learn what water looks like from a target with a 2% positive rate, and the imagery reaches only 9 of the 51 nodes at a 12-day revisit.

8 Leakage: the strongest methodological finding

We ran the identical model under a *random* train/test split instead of a chronological one:

M3 under	**PR-AUC**	**Event detection**
temporal split (honest)	0.7039	**0.380**
random split (leaky)	**0.9050**	**0.089**

A random split makes the model look **29% better** while making it **four times worse** at the job it exists to do. Day 3 of a flood ends up in training and day 4 in test, so the model memorises rather than forecasts.

Leakage does not merely inflate scores — it inverts model selection. Anyone tuning on the random split would choose the wrong model.

9 What is not yet established

Stated plainly, because a result you cannot defend is worse than no result.

1. **Rungs N0–N4 are single-seed.** Across N5_bce's five seeds, the best validation episode PR-AUC ranged 0.3375–0.3856 — a spread of 0.048. So the +0.152 from periodic embeddings is safe (an order of magnitude larger), and the +0.051 from the loss is safe (both sides are 5-seed), but the +0.013 from cross-feature attention and the +0.009 from the ensemble **should not be claimed** without per-seed error bars.
2. **The ev.det column is not comparable across rows** at differing false alarm rates. It needs re-thresholding to a common FAR.
3. **The SAR encoder pretraining has never completed.** Three separate bugs, all now fixed but not yet re-run, so "can a CNN see flooding in Sentinel-1 at all?" is formally untested. Our null result is about *fusion*, not about vision in principle.
4. **No embargo at the split boundary.** Targets look up to 3 days forward, so a training day at the end of 2017 carries a label determined by discharge in early 2018. A handful of rows, but worth checking in a project whose headline finding is about leakage.
5. **Only one protocol for Model 2.** Model 1 was run under temporal, basin-holdout and random splits; Model 2 only under temporal.

10 Summary

1. We built a graph neural network (Model 1), found it lost to a gradient-boosted-tree baseline by 0.108 PR-AUC, and diagnosed this as the known failure of neural networks on tabular data.
2. We built a *graph-free* transformer (Model 2) whose defining feature is that each input number gets its own learned periodic embedding. That single change gained **+0.152 PR-AUC** — the largest effect in the project.
3. We found the focal loss, adopted by default in both ladders as the standard fix for class imbalance, was **actively harmful**: swapping it for plain BCE gained another **+0.051** and cut calibration error to a third.

4. The final model, **N5_bce**, reaches **PR-AUC 0.8355** at 711k parameters — beating Model 1 by 0.093, clearing the operational discharge-percentile rule (0.8164), and closing **87%** of the gap to the tree baseline.
5. Along the way we produced three defensible negative results (message passing, focal loss, satellite imagery) and one methodological finding (leakage inverts model selection) that we consider the most valuable output of the project.